# Projet 4 : Validation croisée et Random Forests

Margot Laignel-Arthur Hauguel-Loïc Mémeteau

## Introduction

L'objectif est d'évaluer des méthodes de classification pour distinguer deux classes de bactéries (`0` et `1`) à partir de signaux spectraux.
On compare une Random Forest et un KNN, avec une validation croisée stratifiée à 10 folds et trois métriques : `accuracy`, `sensitivity` et `specificity` (classe positive = `1`).

## Description des données

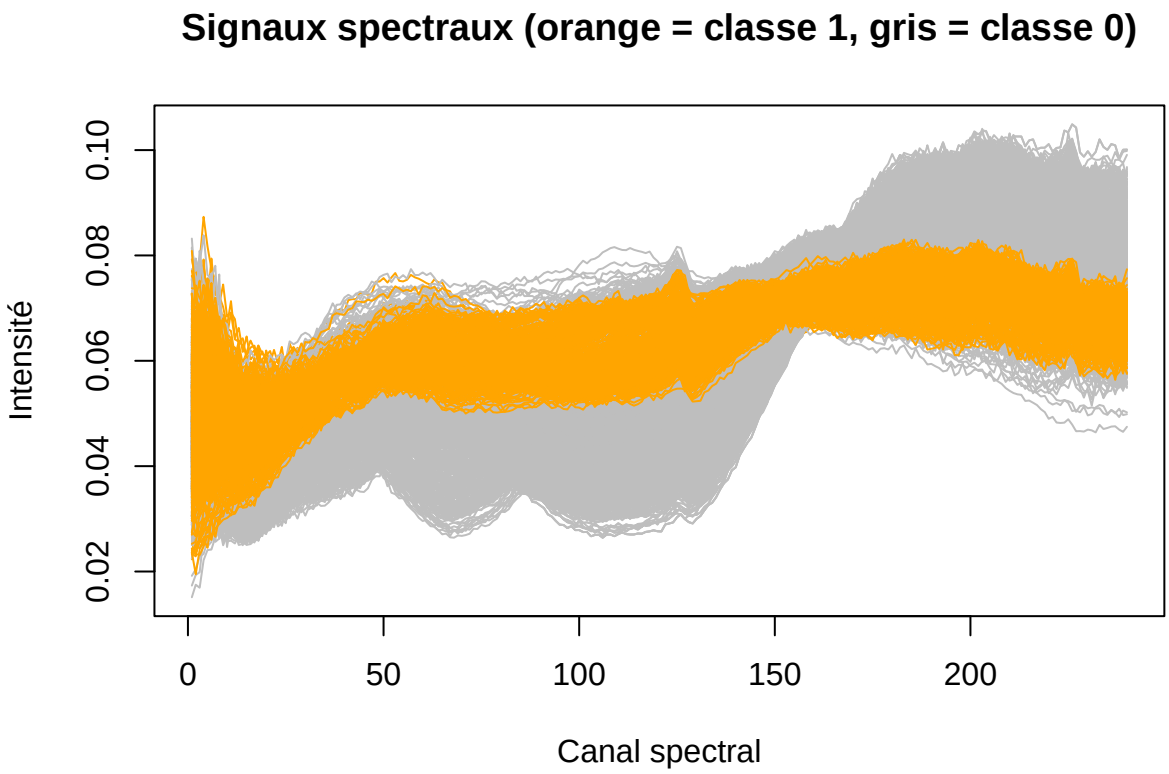
Le jeu contient 2403 observations et 240 variables spectrales.
La classe positive est `1`, la classe négative est `0`.

Table 1: Résumé des dimensions du jeu de données

| n_observations | n_variables | n_labels |
|---|---|---|
| 2403 | 240 | 2403 |

Table 2: Répartition des classes

| classe | effectif | proportion |
|---|---|---|
| 0 | 1839 | 0.7653 |
| 1 | 564 | 0.2347 |




## Méthodologie

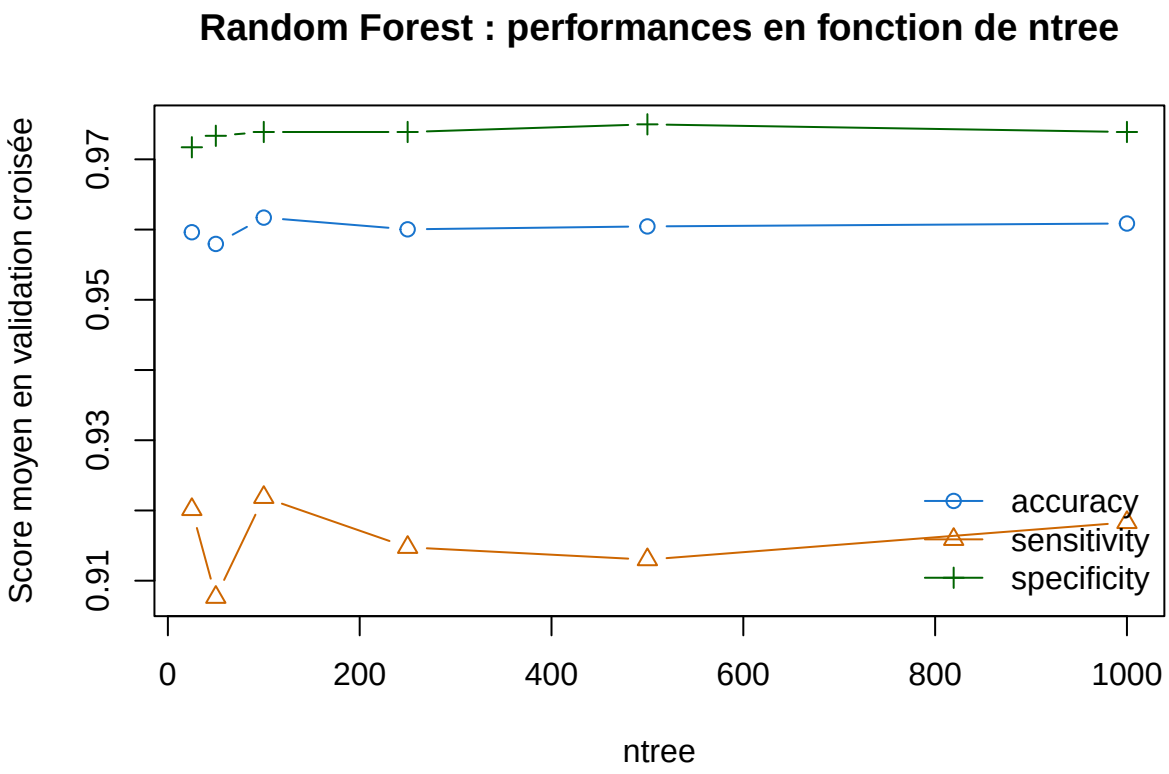
La Random Forest est une agrégation d'arbres aléatoires.
Le paramètre `ntree` fixe le nombre d'arbres. Le paramètre `mtry` contrôle le nombre de variables candidates à chaque split et agit sur la diversité des arbres.
En classification, la valeur par défaut vaut `floor(sqrt(p))`, ici `floor(sqrt(240)) = 15`.

Les performances sont estimées par validation croisée stratifiée à 10 folds avec calcul manuel de la matrice de confusion, puis de `accuracy`, `sensitivity` et `specificity`.

## Résultats Random Forest

Table 3: Performances moyennes de Random Forest selon ntree

| | ntree | accuracy | sensitivity | specificity |
|---|---|---|---|---|
| 25 | 25 | 0.9596 | 0.9201 | 0.9717 |
| 50 | 50 | 0.9580 | 0.9076 | 0.9734 |
| 100 | 100 | 0.9617 | 0.9219 | 0.9739 |
| 250 | 250 | 0.9600 | 0.9148 | 0.9739 |
| 500 | 500 | 0.9604 | 0.9130 | 0.9750 |
| 1000 | 1000 | 0.9609 | 0.9183 | 0.9739 |

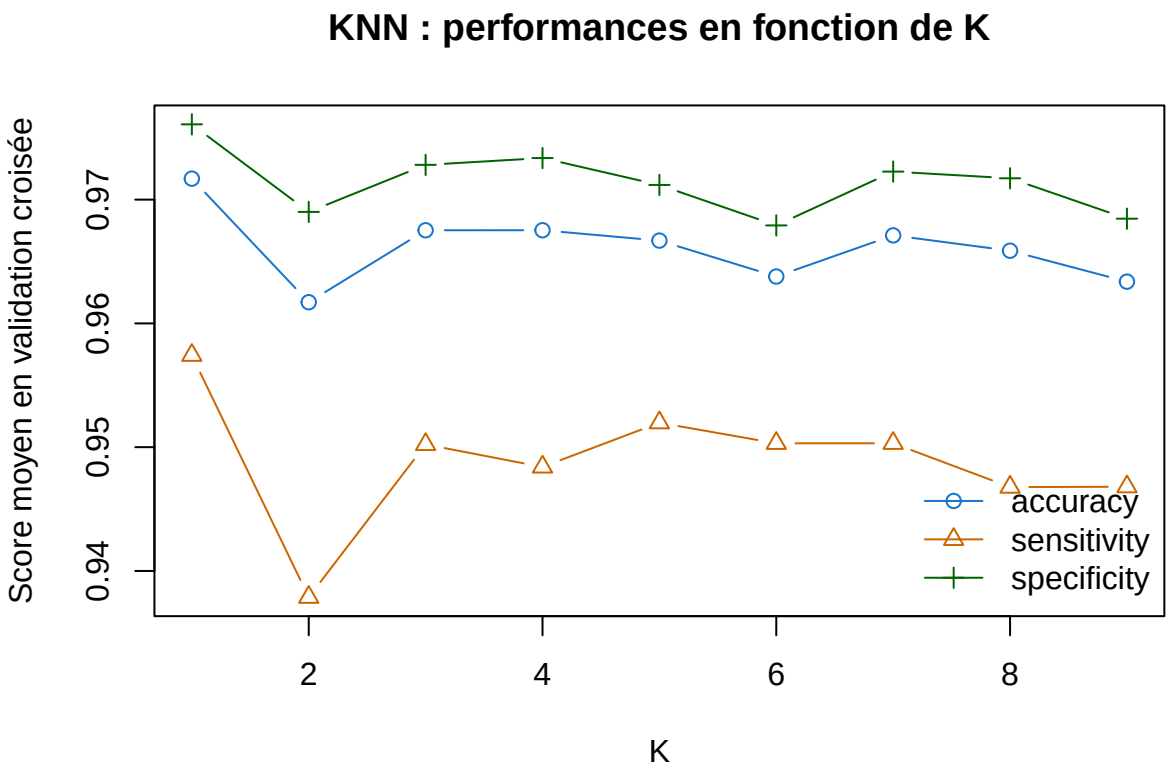



Le meilleur `ntree` (critère `accuracy`) est **100** avec : - accuracy = **0.9617**, - sensitivity = **0.9219**, - specificity = **0.9739**.

## Résultats KNN

Table 4: Performances moyennes de KNN selon K

| K | accuracy | sensitivity | specificity |
|---|---|---|---|
| 1 | 0.9717 | 0.9574 | 0.9761 |
| 2 | 0.9617 | 0.9379 | 0.9690 |
| 3 | 0.9675 | 0.9502 | 0.9728 |
| 4 | 0.9675 | 0.9484 | 0.9734 |
| 5 | 0.9667 | 0.9520 | 0.9712 |
| 6 | 0.9638 | 0.9503 | 0.9679 |
| 7 | 0.9671 | 0.9503 | 0.9723 |
| 8 | 0.9659 | 0.9468 | 0.9717 |
| 9 | 0.9634 | 0.9468 | 0.9685 |

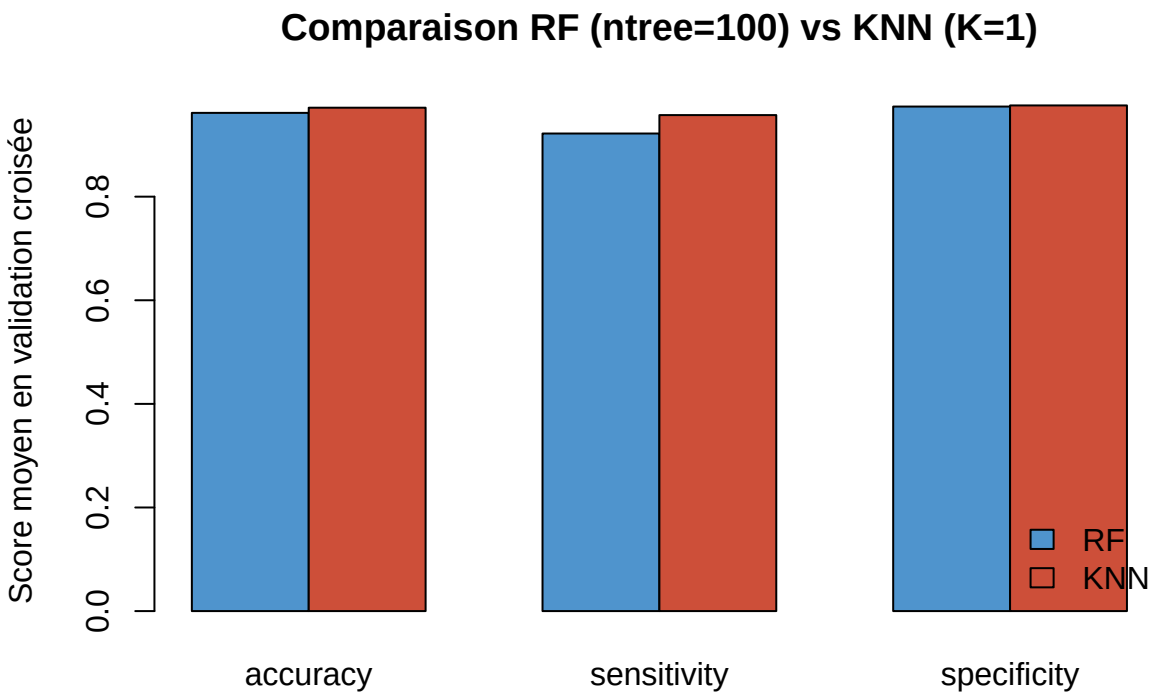



Le meilleur `K` (critère `accuracy`) est **1** avec : - accuracy = **0.9717**, - sensitivity = **0.9574**, - specificity = **0.9761**.

## Comparaison RF vs KNN

Table 5: Comparaison RF vs KNN (meilleurs hyperparamètres)

| | accuracy | sensitivity | specificity |
|---|---|---|---|
| RF | 0.9617 | 0.9219 | 0.9739 |
| KNN | 0.9717 | 0.9574 | 0.9761 |



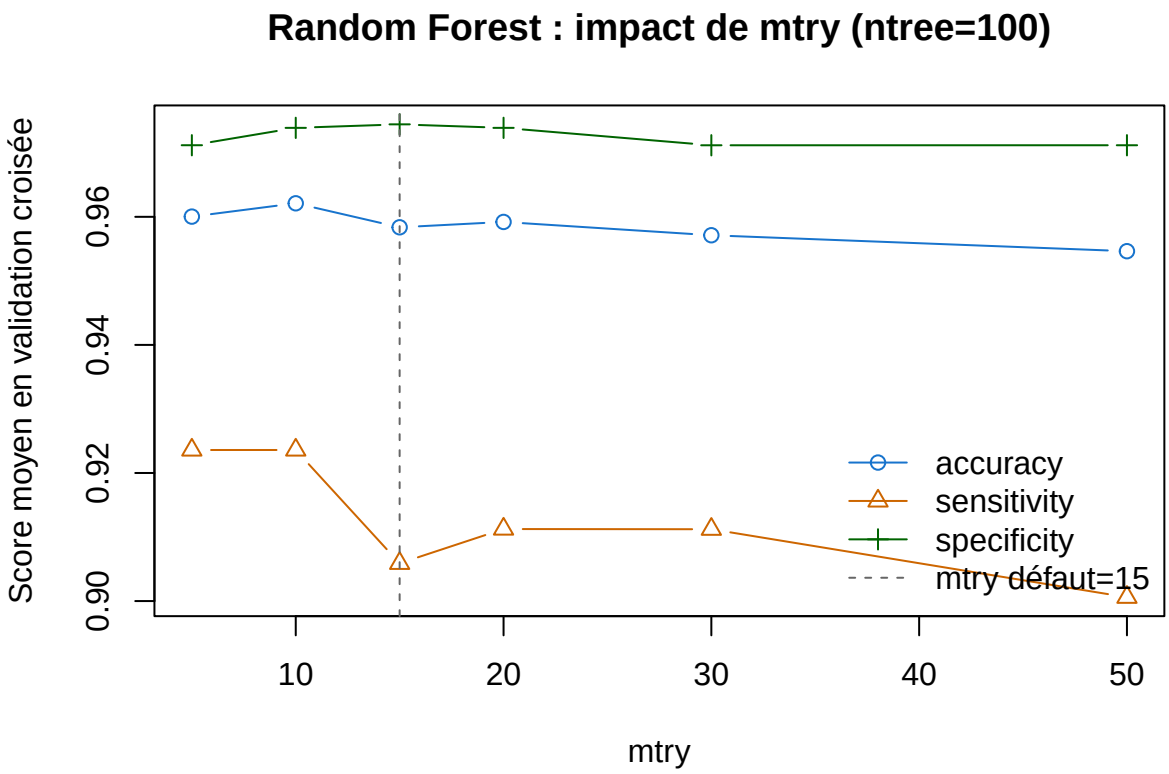

Différences RF - KNN : - accuracy = **-0.0100**, - sensitivity = **-0.0356**, - specificity = **-0.0022**.

## Analyse de mtry

L'étude de `mtry` est détaillée ici en fixant `ntree = 100`.

Table 6: Impact de mtry à ntree fixé

| | mtry | accuracy | sensitivity | specificity |
|---|---|---|---|---|
| 5 | 5 | 0.9600 | 0.9236 | 0.9712 |
| 10 | 10 | 0.9621 | 0.9236 | 0.9739 |
| 15 | 15 | 0.9584 | 0.9059 | 0.9744 |
| 20 | 20 | 0.9592 | 0.9112 | 0.9739 |
| 30 | 30 | 0.9571 | 0.9112 | 0.9712 |
| 50 | 50 | 0.9546 | 0.9006 | 0.9712 |




Le meilleur `mtry` testé est **10** (`mtry` par défaut = 15) avec accuracy = **0.9621**.

## Conclusion

La validation croisée à 10 folds permet d'identifier des hyperparamètres performants pour RF (`ntree`) et KNN (`K`) en se basant sur des métriques de classification calculées manuellement.
La comparaison des deux méthodes met en évidence les écarts sur chaque métrique, et l'analyse de `mtry` précise la sensibilité de Random Forest à ce paramètre. Les résultats nous montrent que le KNN(1) est meilleur sur cette tache que le RF(15) et RF(10).

## Annexe : Code

Conformément à la consigne, le code utilisé est listé ci-dessous.

```
# ---- code_setup ----
knitr::opts_chunk$set(message = FALSE, warning = FALSE)

set.seed(123)
library(randomForest)
load("bacteria-dataset.Rdata")

parametres_ntree <- c(25, 50, 100, 250, 500, 1000)
parametres_k <- 1:9
kfolds <- 10
default_mtry <- floor(sqrt(ncol(X)))
```

```r
# ---- code_metrics ----
accuracy <- function(true_positive, true_negative, false_positive, false_negative) {
  total <- true_positive + true_negative + false_positive + false_negative
  if (total == 0) return(NA_real_)
  (true_positive + true_negative) / total
}

sensitivity <- function(true_positive, false_negative) {
  denom <- true_positive + false_negative
  if (denom == 0) return(NA_real_)
  true_positive / denom
}

specificity <- function(true_negative, false_positive) {
  denom <- true_negative + false_positive
  if (denom == 0) return(NA_real_)
  true_negative / denom
}

results <- function(y_true, y_pred) {
  y_true <- as.character(y_true)
  y_pred <- as.character(y_pred)

  true_positive <- sum(y_true == "1" & y_pred == "1")
  false_positive <- sum(y_true == "0" & y_pred == "1")
  true_negative <- sum(y_true == "0" & y_pred == "0")
  false_negative <- sum(y_true == "1" & y_pred == "0")

  c(
    accuracy = accuracy(true_positive, true_negative, false_positive, false_negative),
    sensitivity = sensitivity(true_positive, false_negative),
    specificity = specificity(true_negative, false_positive)
  )
}

# ---- code_folds ----
idx_pos <- sample(which(as.character(Y) == "1"))
idx_neg <- sample(which(as.character(Y) == "0"))

fold_id <- integer(length(Y))
fold_id[idx_pos] <- rep(seq_len(kfolds), length.out = length(idx_pos))
fold_id[idx_neg] <- rep(seq_len(kfolds), length.out = length(idx_neg))

folds <- lapply(seq_len(kfolds), function(f) which(fold_id == f))

# ---- code_rf_cv ----
rf_perf <- matrix(0, nrow = length(parametres_ntree), ncol = 3)
colnames(rf_perf) <- c("accuracy", "sensitivity", "specificity")
rownames(rf_perf) <- parametres_ntree

for (i in seq_along(parametres_ntree)) {
  n <- parametres_ntree[i]
```

```r
  scores_folds <- sapply(seq_len(kfolds), function(f) {
    test_idx <- folds[[f]]
    train_idx <- setdiff(seq_len(nrow(X)), test_idx)

    mod <- randomForest(
      x = X[train_idx, , drop = FALSE],
      y = as.factor(Y[train_idx]),
      ntree = n
    )
    preds <- predict(mod, X[test_idx, , drop = FALSE])
    results(Y[test_idx], preds)
  })

  rf_perf[i, ] <- rowMeans(scores_folds)
}

# ---- code_knn_cv ----
knn_perf <- matrix(0, nrow = length(parametres_k), ncol = 3)
colnames(knn_perf) <- c("accuracy", "sensitivity", "specificity")
rownames(knn_perf) <- parametres_k

for (i in seq_along(parametres_k)) {
  k <- parametres_k[i]

  scores_folds <- sapply(seq_len(kfolds), function(f) {
    test_idx <- folds[[f]]
    train_idx <- setdiff(seq_len(nrow(X)), test_idx)

    x_train <- scale(X[train_idx, , drop = FALSE])
    center_train <- attr(x_train, "scaled:center")
    scale_train <- attr(x_train, "scaled:scale")
    scale_train[scale_train == 0] <- 1
    x_test <- scale(X[test_idx, , drop = FALSE], center = center_train, scale = scale_train)

    preds <- class::knn(
      train = x_train,
      test = x_test,
      cl = as.factor(Y[train_idx]),
      k = k
    )
    results(Y[test_idx], preds)
  })

  knn_perf[i, ] <- rowMeans(scores_folds)
}

# ---- code_compare ----
max_acc_rf <- max(rf_perf[, "accuracy"])
best_ntree <- min(as.numeric(rownames(rf_perf)[rf_perf[, "accuracy"] == max_acc_rf]))

max_acc_knn <- max(knn_perf[, "accuracy"])
best_k <- min(as.numeric(rownames(knn_perf)[knn_perf[, "accuracy"] == max_acc_knn]))
```

```r
comparaison <- rbind(
  RF = rf_perf[as.character(best_ntree), ],
  KNN = knn_perf[as.character(best_k), ]
)

delta <- comparaison["RF", ] - comparaison["KNN", ]

# ---- code_mtry ----
mtry_values <- sort(unique(c(5, 10, default_mtry, 20, 30, 50)))
mtry_values <- mtry_values[mtry_values >= 1 & mtry_values <= ncol(X)]

mtry_perf <- matrix(0, nrow = length(mtry_values), ncol = 3)
colnames(mtry_perf) <- c("accuracy", "sensitivity", "specificity")
rownames(mtry_perf) <- mtry_values

for (i in seq_along(mtry_values)) {
  m <- mtry_values[i]

  scores_folds <- sapply(seq_len(kfolds), function(f) {
    test_idx <- folds[[f]]
    train_idx <- setdiff(seq_len(nrow(X)), test_idx)

    mod <- randomForest(
      x = X[train_idx, , drop = FALSE],
      y = as.factor(Y[train_idx]),
      ntree = best_ntree,
      mtry = m
    )
    preds <- predict(mod, X[test_idx, , drop = FALSE])
    results(Y[test_idx], preds)
  })

  mtry_perf[i, ] <- rowMeans(scores_folds)
}

max_acc_mtry <- max(mtry_perf[, "accuracy"])
best_mtry <- min(as.numeric(rownames(mtry_perf)[mtry_perf[, "accuracy"] == max_acc_mtry]))

# ---- code_checks ----

# ---- fig_spectres ----
matplot(
  t(X),
  type = "l",
  lty = 1,
  col = ifelse(Y == 1, "orange", "grey"),
  main = "Signaux spectraux (orange = classe 1, gris = classe 0)",
  xlab = "Canal spectral",
  ylab = "Intensité"
)

# ---- fig_rf ----
matplot(
```

```r
  x = parametres_ntree,
  y = rf_perf,
  type = "b",
  pch = 1:3,
  lty = 1,
  col = c("dodgerblue3", "darkorange3", "darkgreen"),
  xlab = "ntree",
  ylab = "Score moyen en validation croisée",
  main = "Random Forest : performances en fonction de ntree"
)
legend(
  "bottomright",
  legend = c("accuracy", "sensitivity", "specificity"),
  col = c("dodgerblue3", "darkorange3", "darkgreen"),
  lty = 1,
  pch = 1:3,
  bty = "n"
)

# ---- fig_knn ----
matplot(
  x = parametres_k,
  y = knn_perf,
  type = "b",
  pch = 1:3,
  lty = 1,
  col = c("dodgerblue3", "darkorange3", "darkgreen"),
  xlab = "K",
  ylab = "Score moyen en validation croisée",
  main = "KNN : performances en fonction de K"
)
legend(
  "bottomright",
  legend = c("accuracy", "sensitivity", "specificity"),
  col = c("dodgerblue3", "darkorange3", "darkgreen"),
  lty = 1,
  pch = 1:3,
  bty = "n"
)

# ---- fig_compare ----
barplot(
  comparaison,
  beside = TRUE,
  col = c("steelblue3", "tomato3"),
  names.arg = colnames(comparaison),
  ylab = "Score moyen en validation croisée",
  main = paste0("Comparaison RF (ntree=", best_ntree, ") vs KNN (K=", best_k, ")")
)
legend(
  "bottomright",
  legend = c("RF", "KNN"),
  fill = c("steelblue3", "tomato3"),
```

```r
  bty = "n"
)

# ---- fig_mtry ----
matplot(
  x = mtry_values,
  y = mtry_perf,
  type = "b",
  pch = 1:3,
  lty = 1,
  col = c("dodgerblue3", "darkorange3", "darkgreen"),
  xlab = "mtry",
  ylab = "Score moyen en validation croisée",
  main = paste0("Random Forest : impact de mtry (ntree=", best_ntree, ")")
)
abline(v = default_mtry, lty = 2, col = "grey40")
legend(
  "bottomright",
  legend = c("accuracy", "sensitivity", "specificity", paste0("mtry défaut=", default_mtry)),
  col = c("dodgerblue3", "darkorange3", "darkgreen", "grey40"),
  lty = c(1, 1, 1, 2),
  pch = c(1, 2, 3, NA),
  bty = "n"
)
```